Before you start, please change the hardware to GPU by clicking on `Runtime` -> `Change runtime type` -> `T4 GPU` -> `save`.

Part 1: Introduction to PyTorch

↳ 107 cells hidden

Part 2: Classification Using Neural Network

We will build neural network models to do classification on the FashionMNIST datset.

Data Preparation

```
print(device)
```

```
cuda
```

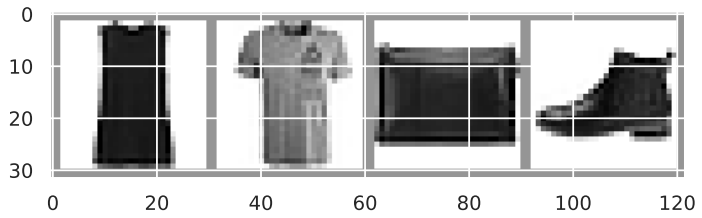
```
import torch
import torchvision
import torchvision.transforms as transforms

# PyTorch TensorBoard support
from torch.utils.tensorboard import SummaryWriter
from datetime import datetime


transform = transforms.Compose(
    [transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))])

# Create datasets for training & validation, download if necessary
training_set = torchvision.datasets.FashionMNIST('./data', train=True, transform=transform, download=True)
validation_set = torchvision.datasets.FashionMNIST('./data', train=False, transform=transform, download=True)

# Create data loaders for our datasets; shuffle for training, not for validation
training_loader = torch.utils.data.DataLoader(training_set, batch_size=4, shuffle=True)
validation_loader = torch.utils.data.DataLoader(validation_set, batch_size=4, shuffle=False)

# Class labels
classes = ('T-shirt', 'Trouser', 'Pullover', 'Dress',
           'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle boot')

# Report split sizes
print('Training set has {} instances'.format(len(training_set)))
print('Validation set has {} instances'.format(len(validation_set)))
```

```
100%|██████████| 26.4M/26.4M [00:01<00:00, 13.2MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 209kB/s]
100%|██████████| 4.42M/4.42M [00:01<00:00, 3.92MB/s]
100%|██████████| 5.15k/5.15k [00:00<00:00, 22.4MB/s]Training set has 60000 instances
Validation set has 10000 instances
```

```
import matplotlib.pyplot as plt
import numpy as np

# Helper function for inline image display
def matplotlib_imshow(img, one_channel=False):
    if one_channel:
        img = img.mean(dim=0)
    img = img / 2 + 0.5     # unnormalize
    npimg = img.numpy()
    if one_channel:
        plt.imshow(npimg, cmap="Greys")
    else:
        plt.imshow(np.transpose(npimg, (1, 2, 0)))
```

```
dataiter = iter(training_loader)
images, labels = next(dataiter)

# Create a grid from the images and show them
img_grid = torchvision.utils.make_grid(images)
matplotlib_imshow(img_grid, one_channel=True)
print('              '.join(classes[labels[j]] for j in range(4)))
```

```
T-shirt              T-shirt              Bag              Ankle boot
```




```
images.shape
```

```
torch.Size([4, 1, 28, 28])
```

The code above prints the shape of the image data. Each batch contains 4 images, where each image contain 1 channel (gray-scale) of 28x28 data.

Single-Layer NN

Let's first create a single-layer neural network model.

This model only has one fully connected layer.

The image size is 28x28, thus the input size of the model must be 28x28.

Since we have 10 classes in the FashionMNIST dataset, the output size of the model must be 10.

```
import torch.nn as nn
import torch.nn.functional as F

# Definition of the single-layer model
class SingleLayerClassifier(nn.Module):
    def __init__(self):
        super(SingleLayerClassifier, self).__init__()
        self.fc1 = nn.Linear(28*28, 10) # fully-connected layer

    def forward(self, x):
        x = x.view(-1, 28*28) # Convert the shape of the input data to 1-D.
        x = self.fc1(x)
        return x

# Create a single layer model. Transfer the model to GPU if available.
model = SingleLayerClassifier().to(device)
```

We will use cross entropy as the loss function.

```
loss_fn = torch.nn.CrossEntropyLoss()
```

Create the SGD optimizer:

```
# Optimizers specified in the torch.optim package
optimizer = torch.optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
```

Here we define a function to train the model in one epoch.
This function will be invoked `num_epoch` times during training.

```
def train_one_epoch():
    epoch_loss = 0.0
    batch_loss = 0.0
```

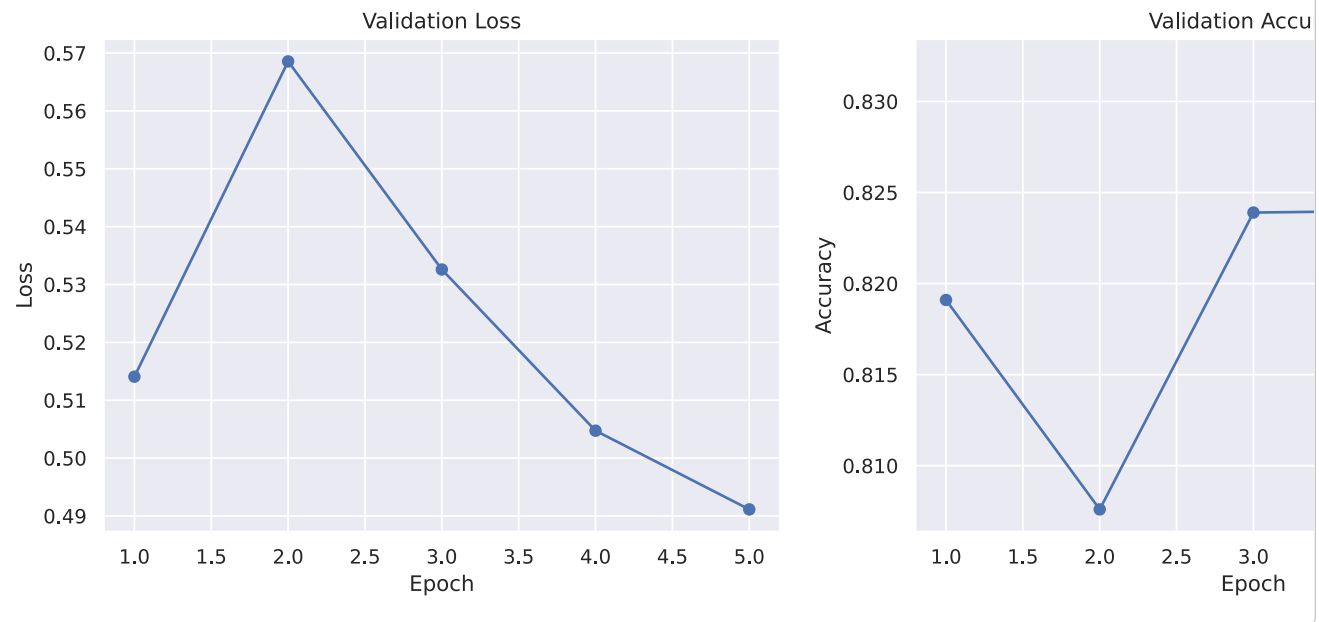
```
    # Here, we use enumerate(training_loader) instead of
    # iter(training_loader) so that we can track the batch
    # index and do some intra-epoch reporting
    for i, data in enumerate(training_loader):
        # Every data instance is an input + label pair
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device) # Transfer to GPU

        # Zero your gradients for every batch!
        optimizer.zero_grad()

        # Make predictions for this batch
        outputs = model(inputs)

        # Compute the loss and its gradients
        loss = loss_fn(outputs, labels)
        loss.backward()

        # Adjust learning weights
        optimizer.step()

        # Gather data and report
        epoch_loss += loss.item()
        batch_loss += loss.item()
        if i % 1000 == 999:
            last_loss = batch_loss / 1000 # loss per batch
            print('  batch {} loss: {:0.6f}'.format(i + 1, last_loss))
            batch_loss = 0.

    avg_loss = epoch_loss / len(training_loader)

    return avg_loss
```

```
# Initializing in a separate cell so we can easily add more epochs to the same run
epoch_number = 0
EPOCHS = 5
valid_loss_history = []
accuracy_history = []

for epoch in range(EPOCHS):
    print('EPOCH {}:'.format(epoch_number + 1))

    # Make sure gradient tracking is on, and do a pass over the data
    model.train(True)

    train_loss = train_one_epoch()


    running_valid_loss = 0.0
    # Set the model to evaluation mode, disabling dropout and using population
    # statistics for batch normalization.
    model.eval()

    # Disable gradient computation and reduce memory consumption.
    correct = 0
    with torch.no_grad():
        for i, vdata in enumerate(validation_loader):
            vinputs, vlabels = vdata
            vinputs, vlabels = vinputs.to(device), vlabels.to(device) # Transfer to GPU
            voutputs = model(vinputs)
            vloss = loss_fn(voutputs, vlabels)
            running_valid_loss += vloss.item()
            _, predicted = torch.max(voutputs, 1)
            correct += (predicted == vlabels).sum().item()

    valid_loss = running_valid_loss / (i + 1)
    validation_accuracy = correct / len(validation_set)
    valid_loss_history.append(valid_loss)
    accuracy_history.append(validation_accuracy)
    print('LOSS train {:.6f} valid {:.6f}'.format(train_loss, valid_loss))
    print('ACCURACY valid {:.6f}'.format(validation_accuracy))

    epoch_number += 1
```

```
EPOCH 1:
  batch 1000 loss: 0.769013
  batch 2000 loss: 0.607334
  batch 3000 loss: 0.556276
  batch 4000 loss: 0.531379
  batch 5000 loss: 0.558113
  batch 6000 loss: 0.534090
  batch 7000 loss: 0.513881
  batch 8000 loss: 0.472908
  batch 9000 loss: 0.529858
  batch 10000 loss: 0.545628
  batch 11000 loss: 0.477181
  batch 12000 loss: 0.505191
  batch 13000 loss: 0.487767
  batch 14000 loss: 0.504294
  batch 15000 loss: 0.487504
LOSS train 0.538694 valid 0.514063
ACCURACY valid 0.819100
EPOCH 2:
  batch 1000 loss: 0.488598
  batch 2000 loss: 0.470215
  batch 3000 loss: 0.468650
  batch 4000 loss: 0.431659
  batch 5000 loss: 0.486306
  batch 6000 loss: 0.511741
  batch 7000 loss: 0.471107
  batch 8000 loss: 0.517632
  batch 9000 loss: 0.451553
  batch 10000 loss: 0.490505
  batch 11000 loss: 0.490055
  batch 12000 loss: 0.463226
  batch 13000 loss: 0.492863
  batch 14000 loss: 0.450469
  batch 15000 loss: 0.466219
LOSS train 0.476720 valid 0.568552
ACCURACY valid 0.807600
EPOCH 3:
  batch 1000 loss: 0.443768
  batch 2000 loss: 0.460582
  batch 3000 loss: 0.461639
  batch 4000 loss: 0.483214
  batch 5000 loss: 0.480062
  batch 6000 loss: 0.495376
  batch 7000 loss: 0.433887
  batch 8000 loss: 0.447126
  batch 9000 loss: 0.447569
  batch 10000 loss: 0.445059
  batch 11000 loss: 0.452849
  batch 12000 loss: 0.484636
  batch 13000 loss: 0.480103
  batch 14000 loss: 0.478283
  batch 15000 loss: 0.447216
LOSS train 0.462758 valid 0.532587
ACCURACY valid 0.823900
EPOCH 4:
  batch 1000 loss: 0.457013
  batch 2000 loss: 0.443111
  batch 3000 loss: 0.461779
```

```
# Plot validation loss and validation accuracy history

fig, axs = plt.subplots(1, 2, figsize=(15, 5))

epoch = np.arange(1, EPOCHS+1)

axs[0].plot(epoch, valid_loss_history, 'o-')
axs[0].set_title('Validation Loss')
axs[0].set_xlabel('Epoch')
axs[0].set_ylabel('Loss')

axs[1].plot(epoch, accuracy_history, 'o-')
axs[1].set_title('Validation Accuracy')
axs[1].set_xlabel('Epoch')
axs[1].set_ylabel('Accuracy')
```

```
Text(0, 0.5, 'Accuracy')
```



Question: Based on the plot, what epoch number gives the best model performance? Justify your answer.

Two-Layer Neural Network

Question: Insert code below to do the following tasks:

Create a neurual network with two layers

The 1st layer is a fully-connected layer with input size 28*28, and output size 100.
The 2nd layer is a fully-connected layer with input size 100, and output size 10.
There is a RELU between the two layers.
For the RELU activation, you may use the built-in function `F.relu()`

```
class TwoLayerClassifier(nn.Module):
    def __init__(self):
        super(TwoLayerClassifier, self).__init__()
        self.fc1 = nn.Linear(28*28, 100) # First fully-connected layer
        self.fc2 = nn.Linear(100, 10)   # Second fully-connected layer

    def forward(self, x):
        x = x.view(-1, 28*28) # Flatten the input
        x = F.relu(self.fc1(x)) # Apply first linear layer and ReLU activation
        x = self.fc2(x)         # Apply second linear layer
        return x

# Create a two-layer model. Transfer the model to GPU if available.
model = TwoLayerClassifier().to(device)
```

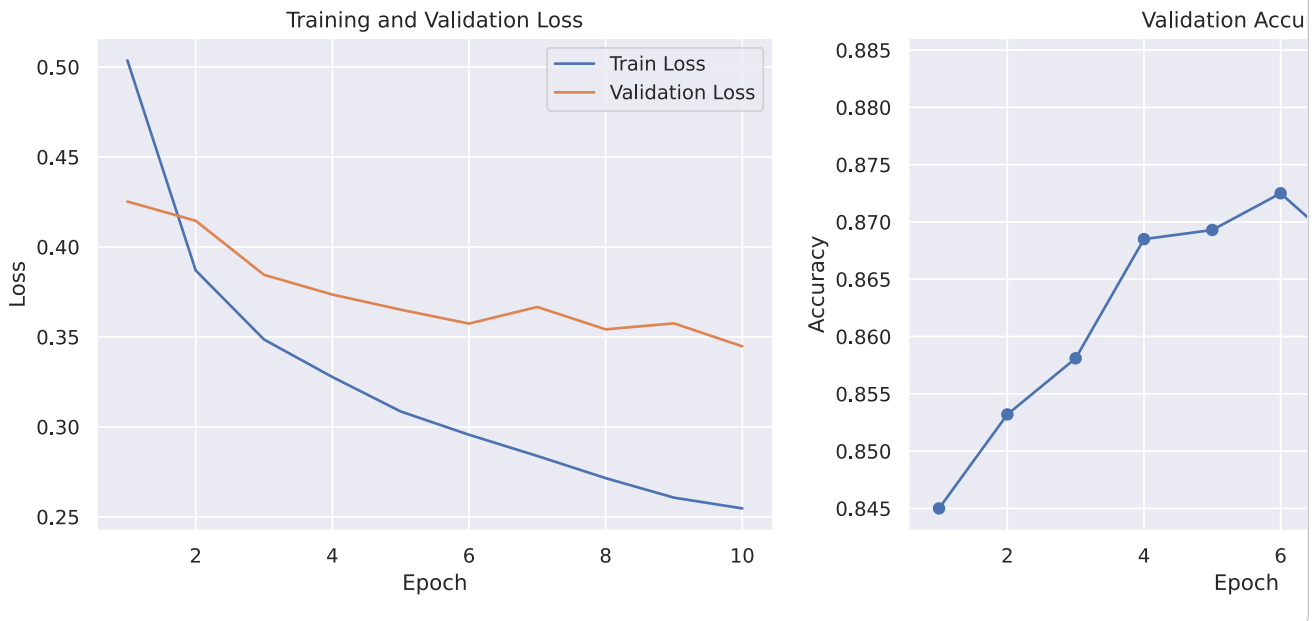
Question: Insert code below to do the following tasks:

1. Create a two-layer classifier model.
2. Create a loss function with cross entropy loss.
3. Create a SGD optimizer, with learning rate 0.001, and momentum 0.9.
4. Train the model with 10 epoch.
5. Plot the validation loss and validation accuracy for each epoch.
6. **Plot the training loss and the validation loss per epoch, in the same figure. (You may need to modify the training code as well, to save a training loss history)**

```
model = TwoLayerClassifier().to(device)
loss_fn = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.001, momentum=0.9)
```

```
def train_one_epoch_two_layer():
    model.train(True)
    epoch_loss = 0.0
    for i, data in enumerate(training_loader):
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = loss_fn(outputs, labels)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()
    return epoch_loss / len(training_loader)

EPOCHS_TWO_LAYER = 10
train_loss_history = []
valid_loss_history_two_layer = []
accuracy_history_two_layer = []

for epoch in range(EPOCHS_TWO_LAYER):
    print('EPOCH {}:'.format(epoch + 1))

    train_loss = train_one_epoch_two_layer()
    train_loss_history.append(train_loss)

    running_valid_loss = 0.0
    model.eval()
    correct = 0
    with torch.no_grad():
        for i, vdata in enumerate(validation_loader):
            vinputs, vlabels = vdata
            vinputs, vlabels = vinputs.to(device), vlabels.to(device)
            voutputs = model(vinputs)
            vloss = loss_fn(voutputs, vlabels)
            running_valid_loss += vloss.item()
            _, predicted = torch.max(voutputs, 1)
            correct += (predicted == vlabels).sum().item()

    valid_loss = running_valid_loss / (i + 1)
    validation_accuracy = correct / len(validation_set)
    valid_loss_history_two_layer.append(valid_loss)
    accuracy_history_two_layer.append(validation_accuracy)
    print('LOSS train {:.6f} valid {:.6f}'.format(train_loss, valid_loss))
    print('ACCURACY valid {:.6f}'.format(validation_accuracy))

fig, axs = plt.subplots(1, 2, figsize=(15, 5))

epoch_range = np.arange(1, EPOCHS_TWO_LAYER + 1)

axs[0].plot(epoch_range, train_loss_history, label='Train Loss')
axs[0].plot(epoch_range, valid_loss_history_two_layer, label='Validation Loss')
axs[0].set_title('Training and Validation Loss')
axs[0].set_xlabel('Epoch')
axs[0].set_ylabel('Loss')
axs[0].legend()

axs[1].plot(epoch_range, accuracy_history_two_layer, 'o-')
axs[1].set_title('Validation Accuracy')
axs[1].set_xlabel('Epoch')
axs[1].set_ylabel('Accuracy')

plt.show()
```

```
EPOCH 1:
LOSS train 0.503635 valid 0.425194
ACCURACY valid 0.845000
EPOCH 2:
LOSS train 0.386893 valid 0.414499
ACCURACY valid 0.853200
EPOCH 3:
LOSS train 0.348515 valid 0.384457
ACCURACY valid 0.858100
EPOCH 4:
LOSS train 0.327680 valid 0.373504
ACCURACY valid 0.868500
EPOCH 5:
LOSS train 0.308535 valid 0.365099
ACCURACY valid 0.869300
EPOCH 6:
LOSS train 0.295591 valid 0.357363
ACCURACY valid 0.872500
EPOCH 7:
LOSS train 0.283801 valid 0.366618
ACCURACY valid 0.867200
EPOCH 8:
LOSS train 0.271479 valid 0.354161
ACCURACY valid 0.877800
EPOCH 9:
LOSS train 0.260634 valid 0.357500
ACCURACY valid 0.876700
EPOCH 10:
LOSS train 0.254662 valid 0.344717
ACCURACY valid 0.884100
```



Question: Based on the plot, what epoch number gives the best model performance? Justify your answer.

ANSWER:

The 10th epoch gives the best performance because it has the lowest train loss and validation loss while also having the highest validation accuracy

CNN

Now, we will try a CNN classifier as follow.

THe model contains two convolution layers, and three fully-connected layers.

Note that convolution and linear layers are always followed by a RELU.

```
import torch.nn as nn
import torch.nn.functional as F

class CNNClassifier(nn.Module):
```

```
    def __init__(self):
        super(CNNClassifier, self).__init__()
        self.conv1 = nn.Conv2d(1, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 4 * 4, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 4 * 4)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x

model = CNNClassifier()
```

Question: Insert code below to do the following tasks:

1. Create and train a CNN model.
2. Plot the training loss, validation loss, and validation accuracy per epoch.
3. Try different number of epochs for the training, until you find the best model.
4. Try a larger learning rate (e.g. 0.005 or 0.01), and get the best number of epoch for the best model.
5. Get the first batch (4 images) of the validation set, plot the image, and print their label.
6. Using your trained model to make a prediction on this batch, print the predicted label.
7. Are the prediction on the first batch correct?

```
model = CNNClassifier().to(device)
loss_fn = torch.nn.CrossEntropyLoss()

def train_one_epoch_cnn(model, optimizer, data_loader, loss_fn):
    model.train(True)
    running_loss = 0.0
    for i, data in enumerate(data_loader):
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = loss_fn(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
    return running_loss / len(data_loader)

def evaluate_cnn_model(model, data_loader, loss_fn):
    model.eval() # Set model to eval mode
    running_valid_loss = 0.0
    correct_predictions = 0
    total_predictions = 0

    with torch.no_grad(): # Deactivate gradients for the following code
        for i, vdata in enumerate(data_loader):
            vinputs, vlabels = vdata
            vinputs, vlabels = vinputs.to(device), vlabels.to(device)
            voutputs = model(vinputs)
            vloss = loss_fn(voutputs, vlabels)
            running_valid_loss += vloss.item()
            _, predicted = torch.max(voutputs, 1)
            total_predictions += vlabels.size(0)
            correct_predictions += (predicted == vlabels).sum().item()

    avg_valid_loss = running_valid_loss / len(data_loader)
    accuracy = correct_predictions / total_predictions
    return avg_valid_loss, accuracy


# Experiment with different learning rates and epochs
learning_rates = [0.001, 0.005, 0.01]
epochs_to_try = [5, 10, 15]

best_accuracy = 0.0
```

```
best_lr = 0
best_epochs = 0
best_model_state = None

for lr in learning_rates:
    for num_epochs in epochs_to_try:
        print(f"\n--- Training CNN with LR: {lr}, Epochs: {num_epochs} ---")
        model = CNNClassifier().to(device) # Re-initialize model for each experiment
        optimizer = torch.optim.SGD(model.parameters(), lr=lr, momentum=0.9)

        current_train_loss_history = []
        current_valid_loss_history = []
        current_accuracy_history = []

        for epoch in range(num_epochs):
            train_loss = train_one_epoch_cnn(model, optimizer, training_loader, loss_fn)
            valid_loss, accuracy = evaluate_cnn_model(model, validation_loader, loss_fn)

            current_train_loss_history.append(train_loss)
            current_valid_loss_history.append(valid_loss)
            current_accuracy_history.append(accuracy)

            print(f"Epoch {epoch+1}/{num_epochs}: Train Loss: {train_loss:.4f}, Valid Loss: {valid_loss:.4f}, Valid Acc: {accur

        # Check if this model is the best so far
        if current_accuracy_history[-1] > best_accuracy:
            best_accuracy = current_accuracy_history[-1]
            best_lr = lr
            best_epochs = num_epochs
            best_model_state = model.state_dict() # Save the state_dict of the best model
            best_train_loss_history = current_train_loss_history
            best_valid_loss_history = current_valid_loss_history
            best_accuracy_history = current_accuracy_history

print(f"\nBest CNN Model Performance: Accuracy = {best_accuracy:.4f} with LR = {best_lr}, Epochs = {best_epochs}")

# Load the best model state
best_model = CNNClassifier().to(device)
best_model.load_state_dict(best_model_state)

# Plotting the results for the best model
fig, axs = plt.subplots(1, 2, figsize=(15, 5))
epoch_range = np.arange(1, best_epochs + 1)

axs[0].plot(epoch_range, best_train_loss_history, label='Train Loss')
axs[0].plot(epoch_range, best_valid_loss_history, label='Validation Loss')
axs[0].set_title(f'Best CNN: Training and Validation Loss (LR={best_lr}, Epochs={best_epochs})')
axs[0].set_xlabel('Epoch')
axs[0].set_ylabel('Loss')
axs[0].legend()

axs[1].plot(epoch_range, best_accuracy_history, 'o-')
axs[1].set_title(f'Best CNN: Validation Accuracy (LR={best_lr}, Epochs={best_epochs})')
axs[1].set_xlabel('Epoch')
axs[1].set_ylabel('Accuracy')

plt.show()

# Get the first batch of the validation set
dataiter = iter(validation_loader)
images, labels = next(dataiter)

# Plot the images and print their true labels
matplotlib_imshow(torchvision.utils.make_grid(images), one_channel=True)
print('True Labels:       ' + ' '.join('%10s' % classes[labels[j]] for j in range(4)))

# Use the trained model to make a prediction on this batch
best_model.eval() # Set model to evaluation mode
with torch.no_grad():
    images = images.to(device)
    outputs = best_model(images)
    _, predicted = torch.max(outputs, 1)

print('Predicted Labels:  ' + ' '.join('%10s' % classes[predicted[j]] for j in range(4)))

# Check if predictions are correct
correct_predictions = (predicted == labels.to(device)).sum().item()
```

```
print(f"\nAre the predictions on the first batch correct? {correct_predictions == len(labels)}")
```

Question:

For your experiments above,

1. What is the best number of epochs learning rate for the CNN?
2. Share your insight regarding model complexity v.s. best number of epochs and learning rate.

ANSWER 1: The best number of epochs and learning rate was 10 epochs with a LR of 0.001

ANSWER 2: The more complex the model, the lower the learning rate needs to be. Higher learning rates lead to lower accuracy.
Additionally, a middle number of epochs is best, as too few result in poor accuracy, but so do too many.

Lab 9 is now complete. Make sure all cells are visible and have been run (rerun if necessary).

The code below converts the ipynb file to PDF, and saves it to where this .ipynb file is.

```
NOTEBOOK_PATH = "/content/drive/MyDrive/ECEN250/ECEN250-F25-Lab9.ipynb" # Enter here, the path to your notebook file, e.g. "/con
! pip install playwright
! jupyter nbconvert --to webpdf --allow-chromium-download "$NOTEBOOK_PATH"
```

```
--- Training CNN with LR: 0.001, Epochs: 5 ---
Epoch 1/5: Train Loss: 0.6069, Valid Loss: 0.4199, Valid Acc: 0.8516
Epoch 2/5: Train Loss: 0.3668, Valid Loss: 0.3600, Valid Acc: 0.8656
Epoch 3/5: Train Loss: 0.3205, Valid Loss: 0.3647, Valid Acc: 0.8585
Epoch 4/5: Train Loss: 0.2949, Valid Loss: 0.3199, Valid Acc: 0.8798
Epoch 5/5: Train Loss: 0.2750, Valid Loss: 0.3141, Valid Acc: 0.8875

--- Training CNN with LR: 0.001, Epochs: 10 ---
Epoch 1/10: Train Loss: 0.6176, Valid Loss: 0.4306, Valid Acc: 0.8431
Epoch 2/10: Train Loss: 0.3749, Valid Loss: 0.3798, Valid Acc: 0.8625
Epoch 3/10: Train Loss: 0.3260, Valid Loss: 0.3370, Valid Acc: 0.8759
Epoch 4/10: Train Loss: 0.2983, Valid Loss: 0.3155, Valid Acc: 0.8863
Epoch 5/10: Train Loss: 0.2792, Valid Loss: 0.3162, Valid Acc: 0.8746
Epoch 6/10: Train Loss: 0.2621, Valid Loss: 0.2885, Valid Acc: 0.8922
Epoch 7/10: Train Loss: 0.2489, Valid Loss: 0.2991, Valid Acc: 0.8905
Epoch 8/10: Train Loss: 0.2385, Valid Loss: 0.2863, Valid Acc: 0.8892
Epoch 9/10: Train Loss: 0.2273, Valid Loss: 0.3037, Valid Acc: 0.8864
Epoch 10/10: Train Loss: 0.2151, Valid Loss: 0.2841, Valid Acc: 0.8918
```

```
Collecting playwright
  Using cached playwright-1.56.0-py3-none-manylinux1_x86_64.whl.metadata (3.5 kB)
Collecting pyee<14,>=13 (from playwright)
  Using cached pyee-13.0.0-py3-none-any.whl.metadata (2.3 kB)
Requirement already satisfied: greenlet<4.0.0,>=3.1.1 in /usr/local/lib/python3.12/dist-packages (from playwright) (3.2.4)
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.12/dist-packages (from pyee<14,>=13->playwright) (4
Using cached playwright-1.56.0-py3-none-manylinux1_x86_64.whl (46.3 MB)
Using cached pyee-13.0.0-py3-none-any.whl (15 kB)
Installing collected packages: pyee, playwright
Successfully installed playwright-1.56.0 pyee-13.0.0
[NbConvertApp] WARNING | pattern '/content/drive/MyDrive/ECEN250/ECEN250-F25-Lab9.ipynb' matched no files
This application is used to convert notebook files (*.ipynb)
        to various other formats.

WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.

Options
=======
The options below are convenience aliases to configurable class-options,
as listed in the "Equivalent to" description-line of the aliases.
To see all configurable class-options for some <cmd>, use:
    <cmd> --help-all
--debug
    set log level to logging.DEBUG (maximize logging output)
--generate-config
    generate default config file
--show-config
    Show the application's configuration (human-readable format)
--show-config-json
    Show the application's configuration (json format)
--y
    Answer yes to any questions instead of prompting.
--execute
    Execute the notebook prior to export.
--allow-errors
    Continue notebook execution even if one of the cells throws an error and include the error message in the cell output (the
```